ChatGPT

LLM Agent Memory: Mechanisms, Architectures, Evaluation, and Open Challenges

Memory Mechanisms in LLM Agents – Taxonomy and Core Components

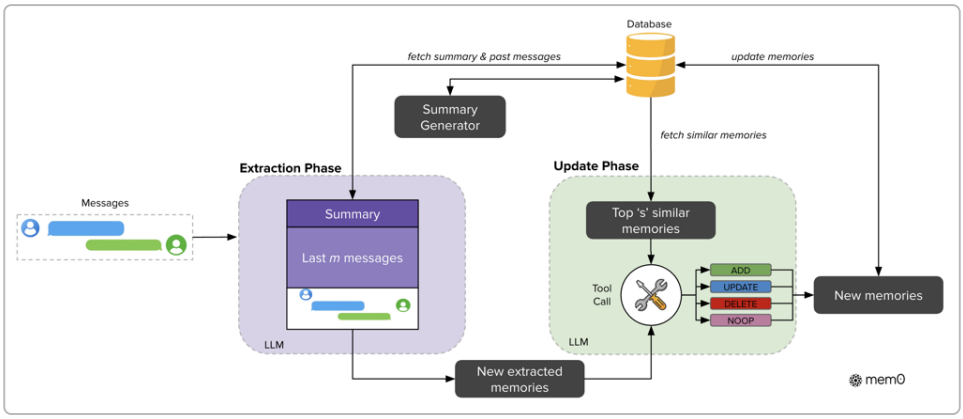
Memory Taxonomy: Large Language Model (LLM) agents employ both **parametric memory** (knowledge encoded in model weights) and **contextual memory** (external, explicit stores) [1] [2]. Contextual memory can be further categorized by source: **short-term or "inside-trial" memory** (recent interaction history within the current session) vs. **long-term or "cross-trial" memory** (knowledge accumulated across sessions) [3] [4]. Many agents also integrate **external knowledge** via tools or APIs (e.g. web search, databases) as an additional memory source beyond the agent's own experiences [5] [6]. In cognitive terms, memory can be organized into **episodic memory** (specific past events or dialogues), **semantic memory** (factual knowledge and learned information), and **procedural memory** (skills or action sequences) [7]. For example, an agent's episodic memory might retain a user's earlier queries or stories, semantic memory might store the user's profile facts or domain knowledge, and procedural memory might store how-to guides or steps for a task [7]. These forms collectively enable an LLM agent to remember **what** happened, **who/what** entities and facts are involved, and **how** to do things.

Memory Operations: To make use of memories, LLM-based agents implement distinct operations for **writing**, **storing/managing**, and **reading** memories [8] [9]. A recent survey defines six fundamental memory operations: **consolidation** (integrating new information into persistent memory), **indexing** (organizing memory for efficient retrieval), **updating** (modifying stored content with new inputs), **forgetting** (removing or de-emphasizing outdated information), **retrieval** (fetching relevant memories when needed), and **compression** (summarizing or compressing memories to save space) [10]. In simpler terms, an LLM agent must be able to **capture** new knowledge, **maintain** and curate a growing knowledge base (through merging, pruning, or re-indexing), and **access** the right pieces of memory at the right time. For instance, when a user provides new information, the agent "writes" it to memory (possibly after extracting key facts); over time it may **manage** the memory by merging duplicate entries or forgetting irrelevant details; and later it **reads** from memory by retrieving stored facts to incorporate into the LLM's context [11] [12]. Many implementations use an **embedding-based vector index** for memory retrieval, where each memory entry is stored with a vector representation and relevant memories are fetched via cosine similarity or other nearest-neighbor search [13] [14]. Others use more structured indices (e.g. keys in a database or nodes in a graph) for memory lookup. The **memory management** component often handles summarization and forgetting policies to cope with memory growth [15] [16]. Overall, this taxonomy of memory types and operations provides a framework to design memory modules systematically [17] [10].

Core Challenges: Equipping LLM agents with long-term memory introduces several challenges. One fundamental issue is the **context length limitation** of LLMs – naively stuffing the entire interaction history into the prompt leads to prohibitively long contexts, quadratic attention costs, and eventual truncation of older content [18] [19]. Even with 100K+ token context windows in new models, the conversation can eventually overflow, and very long prompts suffer from position-bias and reduced attention on earlier tokens [20] [21]. This makes memory management crucial: agents must decide what

to remember explicitly (via memory modules) rather than relying on ever-growing context. Another challenge is **relevance and retrieval accuracy** – the agent's memory system must pull up the right pieces of information at the right time. Inaccurate retrieval can inject irrelevant or incorrect facts into the agent's reasoning [22]. Ensuring that memory search scales efficiently is also difficult; as the volume of stored knowledge grows, vector search or database queries can become a latency bottleneck if not optimized [23] [24]. **Forgetting and consistency** are further issues: the agent should forget or update outdated facts to avoid contradictions, but naive strategies (like always favoring recent data) risk losing important older context [25] [26]. Designing policies for what to forget, when to summarize, and how to reconcile conflicting memories remains an open problem. Finally, an LLM agent's memory must be **robust** – long-term memory can introduce new failure modes such as accumulating errors or biases over time, exposure to adversarial inputs being preserved, and even **self-contamination** (the agent re-consuming its own flawed outputs from memory). In summary, while many approaches to LLM memory exist, they must navigate the trade-offs between **comprehensiveness vs. cost** (remembering enough without overwhelming the model), **stability vs. plasticity** (learning new information without forgetting the old), and **precision vs. recall** (retrieving relevant info without dredging up noise).

Mem0: A Production-Ready Long-Term Memory Framework



Mem0 memory architecture with separate extraction and update phases [27] [28]. The system uses an LLM to extract salient facts from each user-assistant message pair (aided by a running conversation summary and recent message window), then updates a persistent memory store by retrieving similar existing entries and deciding via a 'tool call' whether to ADD new memory, UPDATE an existing record, DELETE a now-contradicted memory, or do nothing (NOOP). Mem0's design thus dynamically grows and curates a knowledge base of the conversation.

Architecture Overview: Mem0 is a scalable memory-centric architecture that addresses LLMs' context limitations by maintaining an external long-term memory and updating it in real time [29] [30]. The Mem0 framework breaks memory processing into two phases (as illustrated above). In the **Extraction Phase**, the agent processes each new interaction (a user query and the assistant's response) to identify salient information worth remembering [31] [27]. To do this in context, Mem0 uses two auxiliary contexts: (1) a conversation summary of the entire dialogue so far, which is asynchronously generated and kept up-to-date, and (2) the last m messages (a recent window) from the conversation [32] [33]. These provide global thematic context and recent detail, respectively. Given the new message pair plus these contexts, an LLM (e.g. a GPT-4-sized model) acts as an **extractor**, producing a set of candidate facts or memory entries (in natural language) that capture the new information relevant to long-term knowledge [34] [33]. In the **Update Phase**, Mem0 takes each extracted candidate memory and checks it against existing memory records to avoid duplication or conflict [35] [36]. Mem0 performs a vector similarity search in its memory database to find the top-s similar stored memories for comparison [37]. Then, using an LLM-based "tool

call" mechanism, it decides one of four operations for each candidate: **ADD** (if it's new information), **UPDATE** (if it refines or extends an existing memory), **DELETE** (if the new info contradicts an old memory), or **NOOP** (if no change is needed) [36]. This decision is made by the LLM reasoning over the candidate and the retrieved similar memories, leveraging the model's understanding instead of a static rule-based matcher [36]. Finally, the chosen operations are applied to the persistent memory store (implemented as a database with vector indexes and optional graph structure) to keep the agent's knowledge base consistent and up-to-date [28] [38]. Mem0 also offers an enhanced variant with **graph-based memory**: in this mode, extracted facts are structured into a knowledge graph of entities and relations (nodes and edges) rather than free-text entries [39] [40]. This graph memory stores, for example, entities like people or items as nodes, and relationships (e.g. "Alice lives_in San_Francisco") as edges, with conflict resolution rules to handle contradictory relations [40] [41]. By organizing facts into a graph, the agent can perform more complex relational reasoning and multi-hop retrieval when answering queries [42] [43].

Scalability and Performance: A key advantage of Mem0's design is that it avoids feeding an ever-growing dialogue into the prompt. Instead, only relevant memories (via retrieval) plus recent context are given to the LLM at any time, dramatically reducing the prompt size for long histories. The Mem0 paper demonstrated significant gains in both **accuracy and efficiency** over other memory solutions. In evaluations on the long-term LoCoMo benchmark (described later), Mem0's memory-augmented agent answered complex questions requiring long dialogue context with higher factual correctness than baseline methods [44] [45]. Notably, Mem0 achieved a **26% relative accuracy improvement** (LLM-as-a-Judge score) over an OpenAI GPT-4 baseline that lacked such a memory module [46]. The graph-augmented Mem0 variant further improved performance on multi-hop and temporal queries, indicating the benefit of structured relational memory [46] [47]. In terms of efficiency, Mem0's selective memory approach yielded a **91% lower 95th-percentile latency** and over **90% reduction in token usage** compared to a naive full-context approach (where the entire dialogue is always fed to the model) [48]. In practice, this means responses were almost instantaneous even as conversation history grew, and computational costs (measured in total tokens processed) were a fraction of the brute-force method [48]. Mem0 achieves this by only injecting a handful of relevant memory snippets into the prompt, rather than hundreds of irrelevant turns [18] [19]. The Mem0 team also open-sourced their system, emphasizing production-readiness features like a persistent database backend, compatibility with high-throughput vector search (FAISS), and integration APIs [49] [50]. In summary, Mem0 exemplifies a **production-grade long-term memory framework**: it dynamically learns from ongoing conversations, uses memory storage and retrieval to extend context, leverages structured knowledge representations, and remains scalable in latency and cost. These traits allowed it to outperform other memory-augmented LLM agents and even proprietary systems on both reasoning quality and efficiency [44] [47].

Evaluating Long-Term Conversational Memory in LLM Agents

As LLM agents gain the ability to accumulate knowledge over many interactions, evaluating how well they remember and use that knowledge becomes crucial. Several benchmarks and evaluation frameworks have been proposed to test an agent's **very long-term memory** and its ability to handle **incremental, multi-turn learning**.

LoCoMo Benchmark – Very Long Conversations: Maharana et al. (2024) introduced **LoCoMo**, a dataset of extremely long multi-session dialogues designed to stress-test an agent's memory [51] [52]. Each LoCoMo conversation spans **up to 35 sessions, ~300 turns, and ~9,000 tokens** on average [53] [54] – far beyond the context length of any current base LLM. These conversations were generated using LLM agents role-playing and grounded in **personas and temporal event graphs**, then validated and edited by humans for consistency [55] [56]. Notably, the dialogues include the agents **sharing and reacting to**

images, making LoCoMo multi-modal (testing memory of visual context as well) [55]. Based on LoCoMo, the authors defined a comprehensive evaluation suite with **three tasks** to probe different aspects of long-term memory [57] [58]:

- **Long-Term Question Answering:** The agent must answer questions about facts or events from earlier in the conversation (potentially many sessions ago). Questions are categorized into reasoning types – single-hop retrieval (a direct fact), multi-hop inference (connecting pieces from different times), temporal reasoning (understanding sequences of events), commonsense/world knowledge (using external knowledge in context), and adversarial (questions designed to confuse or test consistency) [59] [60]. This evaluates how accurately the agent can **recall specific details** from its long-term memory. Performance is measured by factual accuracy (e.g. F1 score for the answer) [61] [62].
- **Event Summarization (Temporal Graph Reconstruction):** The agent is asked to summarize or reconstruct the **event timeline** underlying the conversation [63] [64]. Essentially, given the dialogue, can the model infer the sequence of events or the causal relationships that were discussed? LoCoMo provides ground-truth event graphs (the hidden "life events" for each persona) as reference [58] [65]. This task tests **long-range understanding** – whether the agent grasped the temporal and causal narrative over dozens of interactions. It goes beyond rote recall, requiring the agent to integrate scattered clues from the entire conversation.
- **Multi-Modal Dialogue Generation:** The agent must continue the conversation in a way consistent with earlier contexts, especially with regard to shared images and long-range dependencies [63] [66]. For example, an agent should show awareness of an image shown in Session 1 when it comes up again in Session 5. This evaluates if the agent can produce coherent, contextually grounded responses utilizing long-term memory (including visual memory). Consistency, relevance, and coherence of the generated dialogue are the focus in this task.

Findings on Long-Term Memory: Experiments on LoCoMo revealed that standard LLMs struggle with very long dialogues [67] [68]. Even specialized long-context models (with 100k token windows) and Retrieval-Augmented Generation (RAG) techniques improved performance only modestly and **remained far below human-level** on long-term consistency [69] [56]. For instance, using a 16k-context model or fetching relevant history with a vector database gave 22–66% improvements in QA accuracy over a baseline (showing memory helps), but the models still lagged humans by 56% on average, and by as much as 73% on temporal reasoning questions [61] [62]. Long-context models also exhibited **severe hallucinations** and brittleness: they often produced incorrect answers or got confused by adversarial questions, indicating that simply having a larger context window doesn't guarantee reliable long-term memory use [62] [70]. RAG-based approaches – which simulate memory by retrieving and injecting relevant past dialogue chunks – showed a more balanced performance. In particular, a pipeline that transformed the dialogue into a **database of facts ("observations")** and then retrieved from it (a structured RAG approach) performed best on several tasks [62] [71]. This suggests that organizing memories in a database-like form and doing targeted retrieval can outperform purely long-context attention. Overall, LoCoMo revealed that **temporal coherence** and **causal understanding** over long conversations remain open challenges. Even state-of-the-art models frequently lose track of earlier events or fail to maintain character personas consistently over 35 sessions [67]. The very long-term setting exacerbates issues like forgetting earlier facts, propagating errors, and maintaining a unified narrative.

MemoryAgentBench – Incremental Multi-Turn Memory Evaluation: Hu et al. (2025) proposed **MemoryAgentBench**, a benchmark specifically designed to evaluate LLM-based agents that learn and

update memory incrementally during conversation [72] [73]. They identify four core **memory competencies** that an ideal "memory agent" should exhibit [74] [73]:

- **Accurate Retrieval (AR):** The agent can correctly retrieve and quote the needed information from its memory in response to queries [75] [76]. This tests if the agent remembers facts without distortion – for example, recalling a user's name or a specific preference exactly as given before.
- **Test-Time Learning (TTL):** The agent can **learn new information or behaviors on the fly** during an interaction and use them later, without additional training [76] [75]. In other words, it shows adaptive memory: if a user teaches the agent a new term or a new rule in turn 3, the agent remembers and applies it in turn 10 of the same session (or future session).
- **Long-Range Understanding (LRU):** The agent can **integrate information spread across many turns** (potentially 100k+ tokens apart) to answer questions or make decisions that require a holistic view [77] [78]. This is similar to the LoCoMo objective – ensuring the agent isn't "lost in the details" and can connect distant pieces of context.
- **Conflict Resolution (CR):** The agent can handle conflicting or updated information by **revising or deleting old memories** appropriately [78] [79]. For example, if the user initially said "I live in New York" and later says "Actually I moved to London," a good memory agent will update that fact so it doesn't keep using the old location. This competency aligns with research on model editing and unlearning [78] [79].

MemoryAgentBench repurposes and transforms existing datasets (originally meant for long-context QA or reading comprehension) into a **multi-turn interaction format** where an agent receives information in chunks and is queried incrementally [80] [81]. The authors also created two new datasets, **EventQA** (targeting Accurate Retrieval with story-like data) and **FactConsolidation** (targeting Conflict Resolution by injecting contradictory facts), to ensure all four competencies are covered [82] [83]. Importantly, MemoryAgentBench evaluates not only academic models but also **production systems**: they include tests on commercial memory-augmented agents like Mem0 and open-source agents like MemGPT, as well as simpler baselines (pure context-window agents and basic RAG retrieval agents) [84] [85]. This allows a comparison of how well advanced memory frameworks (like Mem0) actually perform on incremental learning tasks versus simpler approaches. The evaluation uses a consistent protocol across all agents, measuring success on specific query-answer pairs or tasks corresponding to the four competencies [86] [87]. For example, to test TTL, an interaction might "teach" a new fact in one turn and later ask about it; the score reflects if the agent retained that fact. Early results from MemoryAgentBench showed that **no current agent excels at all four competencies** – most have strengths in some areas and weaknesses in others [88] [89]. For instance, a pure RAG agent might do well at accurate retrieval (for factual questions) but fail at conflict resolution (since it doesn't actively delete old info), whereas a sophisticated agent like Mem0 handles updates better but might still struggle on extremely long-range understanding or on-the-fly skill acquisition. This underscores that comprehensive memory in LLM agents remains unsolved, and benchmarks like MemoryAgentBench will help track progress.

Long-Term Memory for Self-Evolution: Beyond user-facing dialog, memory is also crucial for an agent's self-improvement. Jiang et al. (2024) argue that **Long-Term Memory (LTM) is the foundation of AI self-evolution** – enabling an agent to learn continually from interactions instead of remaining static [90] [91]. They outline how an LLM agent with LTM can engage in **lifelong learning**, accumulating experiences and refining its behavior over time [92] [93]. In their work, they present a multi-agent framework called **OMNE** that leverages LTM to allow agents to adapt during inference (not just via offline training). As evidence of the efficacy of this approach, OMNE (with a robust memory system) achieved first place on the **GAIA benchmark** for general AI assistants [94] [95]. GAIA is a rigorous evaluation of multi-step reasoning and learning in agents, co-launched by industry researchers to test real-world problem-solving over extended interactions. OMNE's win suggests that memory-enabled self-evolution gave it an edge, for example by remembering past mistakes and avoiding them in later tasks or by

personalizing to the environment. The AI Self-Evolution report emphasizes several capabilities needed for such agents: a structured LTM store, mechanisms for data retention and representation (e.g. storing interaction logs, outcomes, feedback), and approaches for building **personalized models** from LTM data [91] [96]. They classify methods for continually updating models with new data (while avoiding catastrophic forgetting) and show that these techniques, combined with a long-term memory module, let agents progressively improve performance on tasks through practice [92] [96]. In short, robust long-term memory isn't only about recalling information – it is the key to agents that **learn from experience**. This perspective expands memory evaluation to metrics like how well an agent's performance improves over time or how it adapts to novel situations using accumulated knowledge. Future benchmarks may measure an agent's learning curve across many trials, enabled by memory, which is a different angle than one-off QA accuracy. All these evaluation efforts – from LoCoMo's extremely long chats, to MemoryAgentBench's granular competency tests, to GAIA's focus on learning – highlight that memory is now a first-class citizen in benchmarking AI agents. Going forward, we can expect standardized **"memory tests"** for LLM agents to become as common as logic and reasoning tests, to ensure that as these agents become more persistent and personalized, they do so reliably and safely.

Case Study: Text2SQL Agent with Long-Term User Memory

One concrete application of long-term memory is in **Text2SQL agents** – systems where a user asks questions in natural language and the agent generates SQL queries to retrieve answers from a database. In a production scenario (e.g. a business analytics assistant), it's highly beneficial for the agent to **remember user-specific context and preferences** across sessions [97] [98]. Without memory, each session starts cold, forcing users to repeat definitions or preferences (for example, re-specifying what "high-priority clients" means every time). We examine a production-grade architecture for a Text2SQL agent with persistent user memory (inspired by Mem0) that was detailed in a recent tutorial by MKWriteshere (2025) [99] [100].

Architecture and Components: The Text2SQL agent is designed with a **layered memory system** that separates general conversation understanding from user-specific long-term memory. The core components include:

- **Message Ingestion & Context**: As the user interacts (e.g. asks a database question), the system processes each message pair (user query and the agent's SQL answer). Similar to Mem0, it maintains a **recent message window** and a running **conversation summary** to provide context for memory extraction [27] [34]. This ensures the agent is aware of the immediate dialogue and the overall topic while deciding what to remember.
- **User-Specific Memory Extraction:** The agent has specialized extraction logic targeting **database-related information** in the conversation [101] [102]. Unlike a general chatbot, it looks for four types of information to store for the user:
- Entity Extraction: identifying **database entities or fields** that the user frequently references [103] [104]. For example, noticing that the user often filters on "Region" or talks about "Approved_Loans" table – these entities become part of the user's semantic memory.
- Preference Capture: detecting **user-specific filters or preferences** used in queries [101] [102]. For instance, the user might consistently request "only show approved loans" or "exclude inactive accounts." The agent records these as default preferences tied to the user.
- Terminology Recognition: mapping **user's custom terms to actual database terms** [103] [105]. Users might use domain slang or aliases (e.g. "premium customers" = `Tier_1_Customers` in the DB). The memory stores these mappings so that next time, the agent knows what SQL they correspond to.

- Metric Definitions: storing any **custom calculations or criteria** the user defines [103] [106]. For example, if a user says "high-value sale means deals over \$10k," the agent remembers this definition to apply in future queries.

All these extracted items are essentially building a **user profile knowledge base** (facts about the user's context and preferences) in addition to the dialogue history. By focusing extraction on SQL-relevant aspects, the system keeps the memory targeted and useful for query generation. - **Memory Update & Storage:** The agent uses a mechanism akin to Mem0's update phase: when a new potential memory (entity, preference, etc.) is extracted, it checks the existing store for similar entries (via embedding similarity) and decides whether to **add a new record, update an existing one, or ignore duplicates** [107] [108]. For example, if the user's definition of "high-value sale" changed, the agent would update the stored criterion. This memory store is implemented with a **vector database** (e.g. Postgres with pgVector or an in-memory vector index) for efficient similarity search [101] [109]. Crucially, memory is scoped per user – the architecture ensures **multi-user memory isolation**, meaning each user's data is stored separately to avoid any cross-user leakage [110] [101]. This is important for both personalization and security. The stored memories can be thought of as a set of triples or records like (User=Alice, "approved loans" → filter=`Status='Approved'`), (User=Alice, "luxury properties" → filter=`Property_Value>1M`), etc., along with counters or timestamps for how often they're used. - **Integration with Text2SQL Query Generation:** When the agent receives a new question, before generating SQL it will **retrieve relevant memories** for that user to inform the query. For instance, if the user asks "How many of those were from California?" right after asking about approved loans, the agent's memory recall might bring up (a) the context that "those" refers to approved loans (recent context) and (b) the fact that the user is interested in the `Region='California'` filter (if such preference was noted earlier). The agent then incorporates these into the prompt for the LLM or directly into the SQL generation logic. In practice, the Text2SQL agent connects the memory with the database schema: e.g., knowing that "Region" is a column in the `Orders` table or linking the remembered term "premium customers" to a specific SQL condition [102] [111]. This ensures the retrieved memory is applied correctly in the query. - **User Interface & Feedback Loop:** A production system often includes an interface (the case study used a Gradio web UI) that allows the user not only to query but also to **inspect or modify what the agent has learned** [109] [112]. For example, the agent might show a summary like "I remember: approved loans -> LoanStatus = 'Approved'; luxury properties -> PropertyValue > \$1M" for transparency. The user can correct the agent's memory if needed (this manual feedback acts as additional memory updates).

Benefits in Practice: By implementing these memory components, the Text2SQL agent becomes significantly more **user-friendly and efficient over time**. Persistent memory means that a user's second session of the day can omit redundant details – the agent will recall that "approved loans" should be filtered out, or that "Alice" in the user's query refers to a specific customer ID it saw before. This reduces cognitive load on the user and yields more natural interactions (closer to how one would talk to a human analyst who remembers past conversations) [97] [98]. Moreover, because the memory is tied into the SQL generation, the agent's accuracy improves: fewer queries are missed due to misinterpretation of terms, and fewer errors happen from forgetting a crucial filter. Essentially, the agent builds a **personalized semantic layer** on top of the database. From an engineering perspective, this architecture is **scalable** to many users: each user's memory can be stored as a separate collection in the vector database, and operations (extract, retrieve, update) are done per-user to avoid interference. The use of a database with indexing ensures that even if a user has hundreds of remembered items, retrieving the top relevant ones is fast. The system also addresses data privacy by design (each user's data is isolated and can be wiped on request, etc.) [110]. This Text2SQL memory system is essentially a case study of applying the general Mem0-style approach (extraction + vector store + LLM-based updates) to a specific real-world domain, demonstrating that production-grade long-term memory can be achieved today. No fine-tuning of the LLM was required – it was assembled with prompt engineering and external memory logic – which makes it practical to implement on top of APIs like OpenAI or open-source models.

In summary, the **production Text2SQL agent with long-term memory** shows how memory makes AI assistants more powerful: the agent becomes **stateful** (remembers past interactions), **personalized** (adapts to user's terminology and needs), and **persistent** (accumulates knowledge). This translates to smoother conversations, more accurate database queries, and a system that "learns" from the user, adding real value beyond a stateless SQL generator. It exemplifies how the memory frameworks discussed in surveys can be customized and deployed for industry use-cases.

Comparing Memory Types and Retrieval Strategies

Across the surveyed works and systems, we can identify several **types of memory** that an LLM agent may utilize, as well as different **mechanisms to store and retrieve** those memories:

- **Short-Term vs Long-Term Memory:** All agents have a form of short-term memory – essentially the prompt context or working memory that holds the current conversation turn and recent dialogue. This is transient and limited by the model's context window (e.g. last few thousand tokens). Long-term memory, in contrast, is stored outside the model's immediate context and persists indefinitely (across sessions). Short-term memory provides immediate coherence ("what's happening now?") whereas long-term memory provides continuity over time ("what happened in previous sessions?") [113] [114]. Successful agents combine both: the short-term memory (like a chat history buffer) keeps interactions locally coherent, and the long-term memory is consulted for older facts or seldom-mentioned context. As one source put it, short-term memory makes the agent coherent in the moment, while long-term memory lets it **learn, personalize, and adapt** in the long run [115] [116].
- **Episodic, Semantic, and Procedural Memory:** Many designs draw inspiration from human memory systems by distinguishing **episodic** and **semantic** memory, and sometimes **procedural** memory [7]. **Episodic memory** in agents typically refers to stored records of past events or dialogues (often with time stamps or chronology). For example, an agent might have episodic entries like "Yesterday, User mentioned their hometown is Toronto." **Semantic memory** refers to generalized facts and knowledge an agent has learned – this could include world knowledge (e.g. "Toronto is in Canada") or personal facts about the user ("User's hometown is Toronto"). Over time, episodic memories may be consolidated into semantic form (the agent abstracts general knowledge from specific events) [117] [118]. **Procedural memory** is less discussed but is relevant for agents that perform tasks – it means the agent remembers how to do things or protocols (for instance, the steps to use an API or a recipe the user likes) [7] [119]. In practice, Mem0's layering explicitly included these: episodic memory as a log of interactions, semantic memory as a knowledge base of facts/preferences, and procedural memory as stored how-tos [7]. The benefit of separating memory types is to allow different storage and retrieval methods for each – e.g., episodic might be indexed by time, semantic by topic or keys, procedural by triggers or task names. It also improves interpretability, as the agent (or developers) can inspect "what do we know vs. what have we done vs. what happened when."
- **Textual vs Parametric Memory:** A key distinction highlighted in surveys is between **textual (external) memory** and **parametric (internal) memory** [12] [120]. **Textual memory** refers to information kept outside the model weights – typically as text snippets, embeddings, graphs, or database entries. This is explicit and not bound by the original training data; it can be updated dynamically. **Parametric memory** means the model's weights themselves store knowledge – essentially what the model learned during training or fine-tuning, or via special techniques that allow updating weights with new facts. Textual memory's advantages are that it's **more interpretable** (stored in natural language or structured form) and can be large and growing (only limited by external storage), but it incurs overhead: at query time, those memories must be fetched and fed into the prompt, which has cost and latency implications [11] [12]. Parametric

memory is fast at inference (no need to stuff context – the model "just knows it"), but **updating it is expensive** (requires fine-tuning or model editing) and it might be prone to forgetting details due to compression [120] [121]. In practice, contemporary LLM agents lean heavily on textual/external memory for dynamic and user-specific info – because it's impractical to retrain a model for each user's data. For instance, an agent will not retrain GPT-4 to remember one user's preferences; it will store them in a vector database and retrieve them. On the other hand, parametric memory comes into play with techniques like **knowledge editing** (where a small change is made to the model to inject or remove a fact) [122] [123], or fine-tuning an LLM on logs of an individual agent's interactions to bake in some patterns. A hybrid approach is emerging: some research (e.g. MemoryLLM, M+) uses a **dedicated memory module inside the model** that can absorb long-term context in a compressed way, effectively bridging external and parametric memory [124] [125]. But these are still experimental. The surveys conclude that **textual/external memory is more effective for open-ended conversational tasks** where recent and detailed recall is needed, whereas parametric memory might be better for instilling large bodies of stable knowledge (e.g. an encyclopedia) without running into prompt length limits [126] [120].

- **Retrieval-Augmented Memory vs. Long Context:** Two broad mechanisms for memory retrieval are: (1) **extending the context window** (simply provide the model with as much of the conversation or knowledge base as possible), and (2) **Retrieval-Augmented Generation (RAG)** style (fetch only the most relevant pieces from an external store and inject those into the prompt). Long context usage treats memory like part of the prompt – e.g. models like Claude can directly take 100k tokens of conversation history. This can be powerful for continuity but is computationally expensive (attention on a 100k-token sequence is huge) and still bounded (once you exceed the window, you must drop something) [127] [128]. RAG-based memory is the prevalent strategy: the agent maintains a **memory index** (often a vector index of embeddings for past content) and when needed, it **queries this index** with the current context to get a handful of relevant memory chunks, which are then added to the LLM's input [129] [130]. This is how many open-source "memory" chatbots work: they store the conversation transcript or notes from it, and upon a new user query, they retrieve top-k similar past turns to include. The advantage is unbounded memory size (you can have a million past records, as long as retrieval finds the right few when needed) and efficiency (the prompt stays relatively small). The disadvantage is that if the retrieval fails (misses a relevant fact), the model might act like it forgot even if the info was in memory – because it wasn't surfaced in the prompt. To mitigate this, advanced RAG setups incorporate metadata and sophisticated search criteria (e.g. time-based decay, or separate indices per topic) [131] [13]. Additionally, some systems use a **reflection or summarization** process: rather than keep all raw logs, they periodically summarize old interactions into a concise form, and store those summaries as high-level memory (which can be retrieved for context) [16] [132]. This is akin to a human who vaguely remembers "We discussed project X last month" without recalling the exact words – the summary preserves the essence without cluttering the mind. We saw this in HEMA and other architectures that keep a running summary alongside fine-grained memory [133] [134]. In practice, **state-of-the-art agents use a** hybrid of long-context and retrieval**: for instance, they might utilize the LLM's context window for the very recent dialogue (so coherence is maintained naturally) and use a retrieval system for older knowledge (beyond the window) [135] [114]. This way, the model gets the benefit of both – it "feels" like it has a long memory because older info is injected when needed, but it's not weighed down by irrelevant old text at each turn.
- **Knowledge Graphs vs. Vector Stores:** Another contrast in retrieval mechanisms is **unstructured vs. structured memory**. Most RAG approaches use an unstructured store (chunks of text with embeddings). But some frameworks (like Mem0's graph memory, or the AriGraph system [136] [137]) use a **knowledge graph** where entities and relations are explicitly represented. The benefit of a graph memory is that it enables **symbolic queries and multi-hop traversal**. For example, if an agent is asked "Who is Alice's boss and where did he go to college?", a graph memory can follow links: Alice -> Boss -> Education, to fetch that info, which might be hard to get via pure

vector similarity. Graphs also naturally handle **heterogeneous data** – images, locations, people can all be nodes with different attributes. However, maintaining a graph memory requires additional NLP (to extract entities/relations) and logic for updating edges, which is complex. In contrast, a pure vector store with semantic search is simpler to maintain and can capture implicit relationships via embeddings. In practice, agents like Generative Agents (Park et al.) and Mem0 have shown graphs can improve performance on complex queries [136] [137], at the cost of complexity. A middle ground is storing structured tables or key-value memories (e.g., ChatDB stored dialogue facts in a SQL database and used SQL queries as retrieval) [138]. Such symbolic memory can guarantee precision (no embedding fuzziness) but usually requires the agent to formulate the right query. Overall, we see a spectrum: **raw text memory** (maximal recall, minimal structure), **embedding-indexed memory** (some structure via vector space), **knowledge graphs or databases** (highly structured, requiring proper schema). Each has trade-offs in ease of update, retrieval accuracy, and ability to support reasoning. Modern systems even combine them – e.g. Mem0 uses a text-based vector memory for most information and a graph for specific types of relational data, leveraging both.

- **Episodic vs. Semantic Retrieval:** Related to memory types, agents may retrieve differently depending on whether they need a specific past episode or some general knowledge. For episodic questions ("What exactly did the user say last week about X?"), the agent might pull the verbatim log from that conversation (vector similarity helps find it). For semantic queries ("What's the user's favorite color?"), the agent might retrieve a stored profile fact rather than the entire dialogue where that was mentioned. Some designs explicitly maintain separate indices for these – one index for raw conversation transcripts (episodic memory) and another for distilled facts (semantic memory), querying one or both as needed. For example, **Generative Agents** maintained a list of atomic memory records with importance/relevance scores, and also higher-level reflections – the agent could retrieve raw memories for detail or reflections for summary depending on the question. This improves efficiency and relevance of retrieval.

In summary, LLM-agent memory systems can be compared along several axes. **Memory types:** short-term vs long-term, episodic vs semantic vs procedural, etc., which determine what is stored. **Memory forms:** textual external vs in-model parametric, determining where it's stored. **Memory structure:** unstructured chunks vs structured graphs/tables, determining how it's stored and indexed. **Retrieval method:** exhaustive long-context inclusion vs selective retrieval-augmentation, determining when information is accessed by the model. The trends in the literature favor **hybrid approaches** – combining multiple forms of memory to capture their respective strengths. For instance, an agent might use RAG (vector retrieval) as the backbone to overcome context limits, but also leverage a knowledge graph for complex relations and still rely on the LLM's internal knowledge for common facts. By mixing these, agents aim to achieve both breadth (remember everything important) and depth (reason about it effectively). As research continues, we expect more unified memory frameworks that can juggle different memory types seamlessly – e.g. converting episodic traces into semantic knowledge over time, or balancing between parametric quick recall and explicit memory for detail [139] [140].

Recent Advances and Novel Memory Architectures (2024–2025)

The fast-evolving nature of LLM research in 2024–2025 has produced several new ideas and systems for long-term memory in AI agents, going beyond the methods surveyed in early 2024. Here we highlight some **state-of-the-art techniques and novel memory architectures** that have emerged since the publication of the core papers discussed above:

- **Hippocampus-Inspired Dual Memory (HEMA, 2025):** Drawing an analogy to the human brain's dual memory systems (hippocampus for episodic short-term memories, cortex for long-term storage), HEMA introduces a two-tier memory for LLMs [141] [134]. It maintains a continuously

updated **compact summary** of the dialogue (often just a one-sentence running narrative) alongside a **vector store of detailed episodic chunks** from the conversation [133] [142]. On each turn, the summary is updated to reflect the gist of what happened (preserving global context), while specific past utterances or facts are stored in the vector memory. This way, the agent has a high-level "story so far" always in prompt (which is small) and can retrieve verbatim details from the vector memory when needed. HEMA also implemented **age-based forgetting** – gradually down-weighting or pruning the oldest memory embeddings to keep retrieval efficient [143] [144]. Importantly, HEMA showed strong results: enabling its memory in a chatbot led to big gains in long-term coherence as judged by humans (conversation coherence ratings went up significantly) [145]. An ablation study found that both components – the running summary and the episodic store – were needed for optimal performance [146] [142]. HEMA's approach validates the concept of combining **immediate summarization with persistent storage**, and its use of semantic forgetting is a concrete step to address unbounded memory growth. It echoes strategies in reinforcement learning (experience replay) and cognitive science (memory consolidation) applied to LLM dialogue.

- **MemoryLLM and M+ (Parametric-Plus-External Hybrid):** Researchers have attempted to extend the internal memory of LLMs by adding dedicated "memory neurons" or modules. MemoryLLM (Wang et al. 2024) augmented a 7B model with a ~1B-parameter memory module that learned to **compress past context into latent states** [124] [147]. Essentially, as the conversation went on, it would feed older tokens into this memory module which would condense them, allowing the model to carry some information even beyond its normal context window. While MemoryLLM could retain more than the base model, it was limited (effective up to ~16k tokens of history) [124] [148]. In 2025, IBM's M+ improved this by combining latent memory with retrieval: the model compresses recent history into a latent vector (parametric memory), but for anything older, it uses a learned retriever to fetch from an external store [125] [149]. This hybrid greatly extended retention – experiments showed it could handle contexts of 160k tokens or more by smartly offloading to external memory when needed [125] [150]. The significance of M+ is that it blurs the line between parametric and textual memory: the model itself "decides" when to encode information internally vs. when to rely on external retrieval. Such architectures require complex training (they basically train the model and memory module jointly), but they hint at future LLMs that might come pre-equipped with built-in long-term memory capabilities beyond a fixed context window. For now, these are research prototypes; implementing them in production is non-trivial due to needing custom model training. But they demonstrate a path to **LLMs with expandable memory capacity** through learned compression and retrieval.

- **LongMem (Decoupled Memory Module by Microsoft, 2025):** LongMem proposed keeping the core LLM **frozen** and attaching a separate network for memory handling [130] [151]. In LongMem's design, the LLM acts as a memory encoder (turning text into embeddings) and a separate small model acts as a memory retriever/reader. All past dialogues are stored in a memory bank (vector store), and the retriever network is trained to index and fetch relevant pieces on the fly, feeding them into the LLM's layers. This decoupled approach means the main LLM doesn't have to be fine-tuned for long contexts; the memory module can be improved independently and can scale to essentially unlimited content [152] [153]. A big advantage is avoiding the "stale knowledge" problem – since the external memory can be updated continuously without retraining the big model, the agent's knowledge stays current [154] [155]. LongMem's philosophy ("treat the LLM as a black box with a smart cache") has influenced many practical systems that do exactly this: log all interactions, embed them, and stuff in relevant ones when needed, while keeping the expensive model unchanged. What LongMem added was a learned retrieval mechanism rather than just raw vector similarity – presumably making it better at picking the most useful memories.

It's a step towards more **intelligent memory retrieval** that could consider context or weighting, rather than brute-force nearest neighbor.

- **Memory Consolidation & Reflection Strategies:** Beyond new architectures, there have been improved techniques for **memory maintenance**. For instance, MemoryBank (2023) introduced a scheduled "**forgetting curve**" update where memory embeddings decay over time, simulating human memory decay and prioritizing recent important memories [156] [157]. This encourages the agent to refresh or rehearse memories to keep them alive. Generative Agents (2023) implemented an autonomous **reflection process**: the agent would periodically pause and synthesize higher-level insights from raw memories (e.g., "I notice I often talk about my friend Kai" as a reflection) [158] [159]. These insights were stored to guide future behavior (like forming plans or consistent persona traits). The field has also seen exploration of **emotional memory**: EmotionalRAG (Huang et al. 2024) modifies retrieval by factoring in the agent's current "mood" or emotional state [160] [161]. The idea is that an agent feeling sad might recall sad memories more easily (similar to humans). In practice, EmotionalRAG used sentiment analysis on the context as part of the retrieval key, so that the tone of conversation biases which memories are retrieved. This is an intriguing step toward agents whose memories aren't just cold facts but carry experiential context (useful for empathetic dialogue or consistent role-play). We're also seeing interest in **spatial and multimodal memory** – e.g., agents that can remember locations or images. An emergent idea is giving agents a "memory palace" or spatial map to store where things happened (spatio-temporal memory) [162] [163], which could help in embodied tasks or games.

- **Structured and Multi-Modal Memory Graphs:** The use of structured memory representations has expanded. Beyond Mem0's graph, AriGraph (Ahn et al. 2024) integrated **semantic and episodic memory into a unified knowledge graph** for an agent in an interactive environment [136] [164]. As the agent explored, it added nodes for new objects or people and edges for relations/events involving them [165]. This allowed the agent to answer complex queries better than if it had just a text log. In 2025, Voyager (Wang et al.) demonstrated an LLM agent in Minecraft that continuously learned skills and recorded them in a **code library** (a form of procedural memory) that it could draw from later [166] [167]. This is like a memory of actions instead of words. Similarly, there's work on **multimodal long-term memory**, where an agent can remember images or sounds over multiple interactions [168] [169]. This often means storing image embeddings and having a strategy to retrieve and re-display or describe images when relevant. We can anticipate "memory graphs" that have not just text nodes, but image nodes, video nodes, etc., enabling richer recalled context for the agent.

- **Nemori (2025) – Self-Organizing Memory:** Nemori is a very recent architecture that tackles two often overlooked issues: the **granularity** of memories and the **autonomy of memory organization** [170] [171]. Rather than treating each dialogue turn or fixed chunk as a memory unit, Nemori uses an **Event Segmentation** approach (inspired by cognitive science) to dynamically decide memory boundaries – essentially letting the agent determine what constitutes a meaningful "episode" or event to store [171] [172]. It then uses a **Predict-Calibrate** loop (inspired by free-energy principle) to continually learn from the difference between its predictions and reality, updating memory representations in a more **learning-driven** way rather than rule-based extraction [171] [172]. In simpler terms, Nemori tries to make the agent's memory formation more autonomous and intelligent, not relying on fixed rules or human-defined chunks. It also emphasizes **online processing** (organizing the memory on the fly as the conversation streams, rather than offline indexing later) [173] [174]. Early results showed Nemori significantly outperforming prior systems on LoCoMo and LongMemEval for longer contexts [175] [176]. This suggests that more adaptive memory systems – ones that can decide "what to remember"

using learned criteria – are a promising direction. Nemori's self-organizing principle may help agents avoid both missing key info and being flooded with trivial details.

- **Massive Context Models (Claude, GPT-4, Gemini):** While not memory architectures per se, it's worth noting the industry push toward models with huge context windows (100k to 1M+ tokens) [127] [177]. Anthropic's Claude 2, OpenAI's 128k-token GPT-4, and Google DeepMind's Gemini (reportedly 1M token context) demonstrate the trend of extending how much raw context an LLM can handle [127] [128]. This can be seen as a brute-force form of memory – allowing the model to directly "remember" a very large conversation without external tools. In practice, however, these large contexts face diminishing returns and the same fundamental issues (once you hit the limit, memory is gone; and attention over extremely long sequences may not yield coherent utilization of all info) [19] [178]. Researchers have observed issues like **Lost-in-the-Middle** (models under-utilize the middle parts of very long input) [179] [180]. Therefore, even with massive contexts, the importance of specialized memory mechanisms (like retrieval or summarization) remains. The future might combine both: extremely long but sparsely utilized context (via smart attention or segmenting) along with explicit memory structures.

In summary, since early 2024 the frontier of LLM memory has progressed on multiple fronts: **architectural hybrids** (combining summaries, vectors, and learned retrieval as in HEMA, M+), **human-inspired strategies** (forgetting schedules, event segmentation in Nemori, dual-memory as in HEMA), **enhanced retrieval** (emotional context, learned retrievers in LongMem), and **structured knowledge integration** (graphs and code libraries). The overarching theme is **making memory more scalable, efficient, and intelligent**. Techniques like HEMA's dual memory show we can keep coherence even as transcripts grow by orders of magnitude. Memory editing and conflict resolution are being built deeply into agent models (so they can correct themselves). And researchers are more explicitly addressing when and how to form memories, not just how to retrieve them. We're also seeing memory being tied to other aspects like emotion and planning, moving towards agents that **"understand" their memories** rather than just store them. As LLM applications proliferate, these advances are gradually making their way into open-source tools and platforms (for example, Mem0's code release, and libraries like LangChain offering memory components). We can expect that late-2025 and beyond will bring even more **neural-symbolic memory hybrids**, possibly LLMs with built-in vector search modules, and better **benchmarks** that drive these innovations (MemoryAgentBench is one such step). The field is moving fast, but the end goal is clear: enable LLM agents to have something akin to a human's long-term memory – that is, **extensive, efficient, and evolvable knowledge over time**.

Open Challenges and Future Directions in LLM Agent Memory

Despite the progress made, there remain significant **open research problems and engineering challenges** in deploying large-scale memory systems for LLM agents. We discuss some of the major ones related to memory management, computational cost, and ethical/societal considerations:

Memory Management & Organization: One ongoing challenge is how to **intelligently manage the growing memory store** of an agent. An agent that interacts for months could accumulate thousands of memory entries – without intervention, this leads to inefficient retrieval and potential error accumulation. Deciding what to store (salient information extraction) and when to discard or compress is non-trivial. Storing everything verbatim is unsustainable, but aggressive pruning might remove useful context. Current solutions like periodic summarization, priority scores for memories, or time-based decay are heuristic and may fail in edge cases. For instance, an agent might summarize away a detail that later turns out to be crucial. More adaptive techniques (like Nemori's learning-based segmentation, or dynamically raising/lowering memory item importance based on usage frequency) are needed to achieve

a human-like balance of retention and forgetting [15] [16]. Another aspect is **consistency and conflict resolution**: if an agent has multiple memories that conflict (perhaps due to user changing info or the agent misremembering), how should it reconcile them? As discussed, some systems use an LLM to decide deletes/updates [36], but as memory scales this could become unreliable or expensive. Ensuring the memory database remains a coherent source of truth is challenging – it may require periodic audits or cross-checks (possibly by another AI or a human-in-the-loop) to eliminate contradictions. Memory consolidation (merging related memories) is also under-explored – beyond simple summaries, can the agent autonomously form higher-level concepts from raw experiences? This moves towards lifelong learning and is both a memory and a learning problem. Research into **hierarchical memory** (like multi-level memory graphs, or memory that forms "concepts") is promising but still early [181] [182]. In short, building a memory that isn't just a dumping ground but a self-organizing knowledge base remains an open challenge. This includes how to handle **multimodal memories** (images, audio) in the same framework as text – making different media memories interoperable and searchable under a unified representation [183].

Scalability and Computational Cost: Memory comes at a cost – in storage, in retrieval time, and in added computation per model inference. One clear issue is the **inference cost when using long contexts or many retrieved memories**. Injecting a large chunk of text from memory into the LLM's prompt makes generation slower and more expensive (due to the quadratic attention mechanism) [20] [184]. While Mem0 and others show that a clever memory system can reduce overall tokens needed (by not repeating the entire history) [48] [48], the worst-case scenario is still heavy: if a user query triggers retrieval of, say, 20 long memory chunks, that could be hundreds or thousands of extra tokens the model must attend to. Techniques like **sparse attention** or chunked processing are being explored to mitigate this, but they add complexity. There's also the **embedding cost**: every time you add a new memory or need to retrieve, you must compute embeddings (usually via a transformer encoder) which is not free. For very fast-paced systems, embedding generation or vector search can become a throughput bottleneck. Caching embeddings and using incremental updates can help. Moreover, as memory grows, even similarity search can slow down if using brute-force; solutions like hierarchical indices, clustering, or approximate nearest neighbor search (ANN) become important to keep retrieval in sub-linear time. Ensuring the memory system scales to potentially millions of entries (for power users or long-lived agents) is an engineering challenge – it might require distributed databases, sharding by topic, and so on. From a research angle, **compression techniques** are vital: how to compress 100k tokens of past chat into a much smaller state that still lets the model answer questions about it? Approaches like M+ that compress into latent vectors are one idea [125] [150], another is recursively summarizing (summaries of summaries). But compression can lead to information loss – an open question is how to guarantee that compressed memory retains the information most likely to be needed. Perhaps future models will incorporate a form of learned compression that is query-aware (i.e., they can generate a condensed memory representation that's good enough to answer future queries, using some training signal). **Latency vs. completeness** is a trade-off: memory lookup should be fast, but we don't want to miss relevant info. Systems might use multi-stage retrieval (quickly narrow down candidates with a cheap method, then refine with a more expensive rerank) to remain swift. Mem0's success partly came from reducing p95 latency by retrieving only a few highly relevant items and not searching an ever-growing context [47] [185]. Agents will likely need to adopt **memory eviction policies** (like cache eviction in CPUs) to throw out or archive very old items to keep working sets small for speed. This intersects with memory management: deciding what to evict for performance while not harming utility. A related engineering challenge is **robustness of memory retrieval** – making sure retrieval doesn't fail silently. If vector search returns nothing useful (due to embeddings drifting or a query outside distribution), the agent might act as if it forgot. Research into **reliable long-term retrieval** includes making embeddings more robust over time (e.g. fine-tuning the embedding model on the agent's dialogue domain) and monitoring retrieval (if no memory has a cosine similarity above a threshold, maybe trigger a different strategy or signal uncertainty).

Alignment, Ethics, and Privacy: Introducing long-term memory into AI agents raises new ethical considerations. One big concern is **user privacy**. A chatbot with memory might store personal details about a user (preferences, identifying info, sensitive conversations). If this memory is not handled carefully, it could leak – either to other users (if memory isn't properly isolated) or via the model itself (the model might accidentally regurgitate someone else's memory if prompts get mixed, etc.). Ensuring **memory isolation** (as in the Text2SQL case) and data encryption/protection in storage is crucial. Additionally, users should have **control over their data**: the right to ask the agent "forget what I just told you" or to delete their conversation history from the memory store [186] [187]. Implementing a "forget my last query" is non-trivial because of backups and logs – but ethically it may be necessary (think of GDPR-style "right to be forgotten"). Another aspect is **consent**: users should know that the agent is remembering things. Ideally, the UI or agent should make it clear ("I'll remember that for next time, unless you prefer I don't"). OpenAI's guidelines for their memory plugins, for example, emphasize transparency and control. There's also the risk of **bias or unfairness**. If an agent remembers a user's past mistakes or emotional moments indefinitely, it might treat them differently (the equivalent of a human holding a grudge or making a biased judgment). Agents need possibly to practice "forgiving" or bias correction – maybe forgetting certain emotional outbursts or giving more weight to recent behavior to avoid locking a user into a label from long ago. Memory could also reinforce **confirmation bias**: if an agent has formed a certain view of the user or topic from early interactions, it might focus on memories that confirm that and ignore contradictory new info, thus not updating properly. This ties into alignment – an agent that self-evolves with memory might drift in behavior. As noted in the second survey, repeated exposure to its own outputs can create a feedback loop that **biases the agent's trajectory** [188] [189]. For instance, if an agent slightly misinterprets a user preference and then keeps remembering it that way, it might amplify the error over time. Ensuring **truthfulness and accuracy** in memories is an open challenge: how do we validate that what the agent "remembers" is actually correct and not a hallucination it accidentally committed to memory? One possible approach is to have guardrails or verification steps when writing to memory (e.g., check statements against a knowledge base), but this is hard in open domains.

Ethically, another aspect is **misuse of long-term memory**. A malicious actor could try to poison an agent's memory with false info or offensive content, knowing it will stick. There need to be moderation and filtering on what gets into long-term memory stores (similar to how we moderate training data). And conversely, an agent with a long memory could potentially be more manipulative or persuasive (since it knows a user's weaknesses or buttons to push). This raises concerns about autonomy and trust – users might feel uncomfortable if a bot "knows" so much about them. Maintaining **user trust** will require careful design: e.g., allowing users to inspect their memory profile ("What do you remember about me?") to correct mistakes or delete things, much like one can download their data from services today.

Finally, **evaluation challenges** themselves are a meta-issue: we need better ways to measure an agent's memory capabilities consistently. As Du et al. (2025) noted, current benchmarks rarely test things like consolidation or forgetting in dynamic settings [190] [191]. A unified evaluation could involve long-running simulations where an agent is scored on maintaining consistency, adaptability, and user satisfaction over time, not just QA accuracy. Without such evaluations, it's hard to quantify progress on memory (which is why MemoryAgentBench and others were created). This is an active area – creating datasets and tests that are realistic (not just synthetic chat) and cover ethical aspects too (e.g., does the agent correctly forget private info when asked).

In conclusion, while LLM agents with memory are becoming a reality, **significant research and engineering work lies ahead** to ensure these memories are used effectively, efficiently, and ethically. Key open problems include: developing **robust memory management algorithms** (for extraction, consolidation, and forgetting) that can handle long-term interactions; balancing **compute costs** by employing intelligent retrieval and compression (perhaps with learned neural components); and

establishing **ethical practices and safeguards** for memory usage (privacy controls, transparency, avoiding harmful biases). The vision of an aligned, continually learning AI assistant – one that remembers past interactions and evolves – depends on solving these challenges. Encouragingly, the community has identified many of them: for example, calls for unified memory frameworks with shared indexing across modalities [183] and multi-agent memory sharing protocols [192] [193] are pushing the envelope. As we develop these solutions, interdisciplinary input from cognitive science (how do humans efficiently remember?), database theory, and ethics will be invaluable. Ultimately, the goal is to build memory systems that are **scalable** (can grow without breaking), **secure** (respect privacy and integrity), and **supportive** of the agent's goals (improving performance and user experience without unintended side effects). Achieving this will be a cornerstone in the journey toward truly intelligent, autonomous agents.

References: The above discussion synthesizes information from recent survey papers on LLM-based agent memory [1] [2] [12] [120], the Mem0 framework for long-term memory [27] [28] [48], evaluation studies on conversational memory [55] [59] [75] [77], a real-world Text2SQL memory architecture [101] [103], and emerging research on advanced memory systems [134] [125] [143] [188]. These sources provide a foundation for understanding the current state and future directions of memory mechanisms in LLM agents.

---

[1] [2505.00675] Rethinking Memory in AI: Taxonomy, Operations, Topics, and Future Directions
https://arxiv.org/abs/2505.00675

[2] [10] [17] [139] [140] [162] [163] [179] [180] [181] [182] [183] [188] [189] [190] [191] [192] [193] [2505.00675] Rethinking Memory in AI: Taxonomy, Operations, Topics, and Future Directions
https://ar5iv.labs.arxiv.org/html/2505.00675v2

[3] [4] [5] [6] [8] [9] [11] [12] [13] [14] [18] [19] [20] [21] [22] [23] [24] [25] [26] [120] [121] [122] [123] [126] [131] [138] [166] [167] [178] [184] [2404.13501] A Survey on the Memory Mechanism of Large Language Model based Agents
https://ar5iv.labs.arxiv.org/html/2404.13501v1

[7] [15] [16] [113] [114] [119] [132] [135] [186] [187] Building Production-Ready AI Agents with Scalable Long-Term Memory: Unlocking the Potential of Mem0 for Smarter, Stateful Intelligence - Lawhustle
https://golawhustle.com/blogs/building-ai-agents-scalable-long-term-memory

[27] [28] [31] [32] [33] [34] [35] [36] [37] [38] [39] [40] [41] [42] [43] [49] [50] [2504.19413] Mem0: Building Production-Ready AI Agents with Scalable Long-Term Memory
https://ar5iv.labs.arxiv.org/html/2504.19413v1

[29] [30] [44] [45] [46] [47] [48] [185] [2504.19413] Mem0: Building Production-Ready AI Agents with Scalable Long-Term Memory
https://arxiv.org/abs/2504.19413

[51] [52] [53] [54] [55] [56] [67] [68] [69] [2402.17753] Evaluating Very Long-Term Conversational Memory of LLM Agents
https://arxiv.org/abs/2402.17753

[57] [58] [59] [60] [61] [62] [63] [64] [65] [66] [70] [71] Evaluating Very Long-Term Conversational Memory of LLM Agents
https://snap-research.github.io/locomo/

[72] [73] [74] [88] [89] [2507.05257] Evaluating Memory in LLM Agents via Incremental Multi-Turn Interactions
https://arxiv.org/abs/2507.05257

75 76 77 78 79 80 81 82 83 84 85 127 128 177 [2507.05257] Evaluating Memory in LLM Agents via Incremental Multi-Turn Interactions
https://ar5iv.labs.arxiv.org/html/2507.05257v2

86 87 GitHub - HUST-AI-HYZ/MemoryAgentBench: Open source code for Paper: Evaluating Memory in LLM Agents via Incremental Multi-Turn Interactions
https://github.com/HUST-AI-HYZ/MemoryAgentBench

90 91 92 93 94 95 96 [2410.15665] Long Term Memory: The Foundation of AI Self-Evolution
https://arxiv.org/abs/2410.15665

97 98 101 102 103 104 105 106 107 108 109 110 111 112 GitHub - MKcodeshere/Text2SQl-Agent-with-Long-Term-Memory
https://github.com/MKcodeshere/Text2SQl-Agent-with-Long-Term-Memory

99 100 Building a Text2SQL Agent With Long-Term Memory: A Production Grade Architecture for User Memory Isolation | by MKWriteshere | Data Science Collective | Medium
https://medium.com/data-science-collective/building-a-text2sql-agent-with-long-term-memory-a-production-grade-architecture-for-user-memory-9d6fc3509e92

115 116 Supercharging AI Agents with Memory on Azure Managed Redis
https://techcommunity.microsoft.com/blog/azure-managed-redis/supercharging-ai-agents-with-memory-on-azure-managed-redis/4457407

117 118 124 125 129 130 133 134 136 137 141 142 143 144 145 146 147 148 149 150 151 152 153 154 155 156 157 158 159 160 161 164 165 Exploration of Modern LLM Memory Structures (GPT enabled review) – The Real Cat AI Labs: Developing morally aligned, self-modifying agents—cognition systems that can reflect, refuse, and evolve
https://therealcat.ai/exploration-of-modern-llm-memory-structures-gpt-enabled-review/

168 169 Multimodal Long-Term Memory - Emergent Mind
https://www.emergentmind.com/topics/multimodal-long-term-memory

170 171 172 173 174 175 176 Nemori: Self-Organizing Agent Memory Inspired by Cognitive Science
https://arxiv.org/html/2508.03341v3